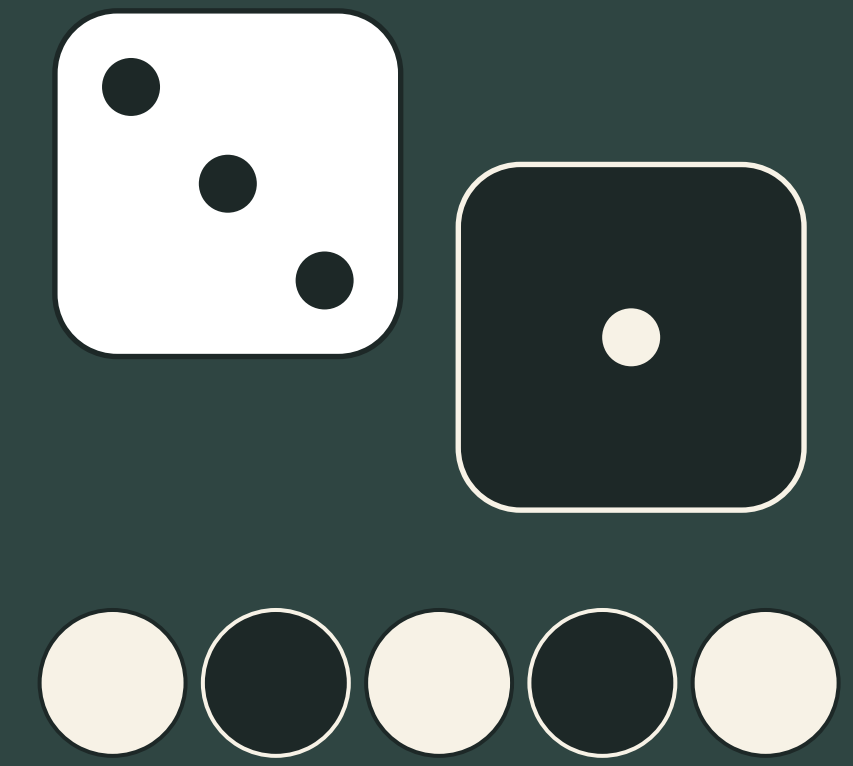
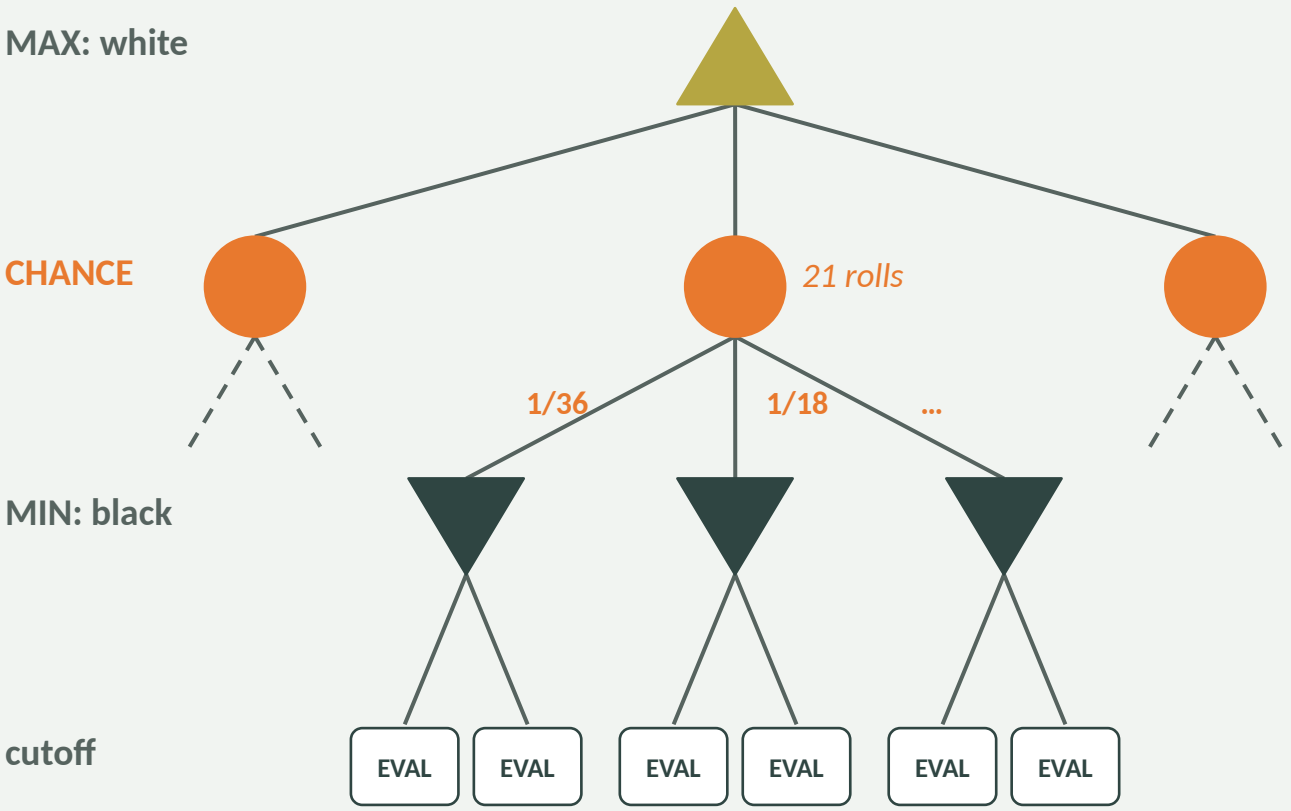
Games with chance

Backgammon in Prolog

Rules as predicates, an EVAL you can tune, and expectiminimax with a cutoff



Where chance enters the tree




```
EXPECTIMINIMAX(n) =
  EVAL(n)          if cutoff(n)
  max over moves   if MAX moves
  min over moves   if MIN moves
  sum of P(r) * value(r)
                   if chance node
```

Course slides M6, 38 to 42. Between two players' decisions there is now a chance node: we cannot choose the roll, only weight it by its probability.

Why we have to cut the search

21

different rolls at every chance node

≈ 21

different moves per roll from the opening (4 to 75)

5,611

positions evaluated to choose one move with 3-1

≈ 2.5 M

if we searched one more chance level

So we do what course slides M6, 31 to 33 propose: search one chance level, then stop and estimate the position with a heuristic EVAL instead of playing the game to the end.

Numbers measured with the code of this presentation, from the opening position.

A position is a Prolog term

```
initial(state([-2,0,0,0,0,5,  0,3,0,0,0,-5,
               5,0,0,0,-3,0, -5,0,0,0,0,2], 0, 0, 0, 0)).
```

```
?- initial(S), show(S).
 13  14  15  16  17  18  | 19  20  21  22  23  24
 W5   .   .   . B3   .  |B5   .   .   .   . W2
 B5   .   .   . W3   .  |W5   .   .   .   . B2
 12  11  10   9   8   7  |  6   5   4   3   2   1
 bar: W0 B0   off: W0 B0
```

1. **Points**

   24 integers: + white, − black.

2. **Bar**

   Checkers of each side on the bar.

3. **Off**

   Checkers each side has borne off.

4. **Moves**

   m(From, To, Die), with bar and off.

One checker, one die

```
% step(+State, +Die, -Move, -NewState): one white checker, one die
step(S, D, m(bar, T, D), S2) :-                  % on the bar: enter first
    S = state(Pts, BW, _, _, _), BW > 0, !,
    T is 25 - D,
    nth1(T, Pts, N), N >= -1,                    % not blocked by 2+ black
    apply_move(S, m(bar, T, D), S2).
step(S, D, m(F, T, D), S2) :-
    S = state(Pts, 0, _, _, _),
    nth1(F, Pts, N), N > 0,                      % a white checker on F
    T0 is F - D,
    (   T0 >= 1
    ->  nth1(T0, Pts, M), M >= -1, T = T0        % land: not blocked
    ;   all_home(S),                             % bear off
        ( T0 =:= 0 ; \+ white_above(Pts, F) ),
        T = off
    ),
    apply_move(S, m(F, T, D), S2).
```

1. On the bar: enter first, at 25 − Die. The cut stops any other move.
2. Land on a point with at most one black checker; if there is one, it is hit.
3. Bear off with an exact die, or a larger one if nothing is above.

Black is white in a mirror

```
1  mirror(state(P, BW, BB, OW, OB), state(M, BB, BW, OB, OW)) :-
       reverse(P, R), maplist(neg, R, M).
   neg(X, Y) :- Y is -X.

   mirror_move(m(F, T, D), m(F2, T2, D)) :- mirror_pt(F, F2), mirror_pt(T, T2).
   mirror_pt(bar, bar) :- !.
   mirror_pt(off, off) :- !.
   mirror_pt(P, Q) :- Q is 25 - P.

2  % single(+State, +Player, +Die, -Move, -NewState)
   single(S, white, D, M, S2) :- step(S, D, M, S2).
   single(S, black, D, M, S2) :-
       mirror(S, W), step(W, D, MW, W2),
       mirror(W2, S2), mirror_move(MW, M).
```

1. Reverse the 24 points and flip the signs: black's position seen as if it were white's.

2. Black moves = white moves on the mirrored board, mirrored back. Only white's rules are written.

Using the dice, one at a time

```
1  dice_orders([D, D], [[D, D, D, D]]) :- !.        % doubles: four moves
   dice_orders([D1, D2], [[D1, D2], [D2, D1]]).

   % use_dice(+State, +Player, +Dice, -Moves, -Final)
2  use_dice(S, _, [], [], S).
   use_dice(S, P, [D|Ds], [M|Ms], F) :-
       single(S, P, D, M, S1),
       use_dice(S1, P, Ds, Ms, F).
3  use_dice(S, P, [D|_], [], S) :-                  % this die cannot be used
       \+ single(S, P, D, _, _).
```

1. Doubles give four moves; otherwise try both orders of the two dice.
2. Move one checker with the first die, then use the rest of the dice.
3. If a die cannot be used, the turn stops there: maybe fewer moves.

The two rules of a whole turn

```
legal_turns(S, P, Dice, Turns) :-
    dice_orders(Dice, Orders),
    findall(L-(F-Ms), ( member(O, Orders),
                        use_dice(S, P, O, Ms, F),
                        length(Ms, L) ), All),
1   aggregate_all(max(L), member(L-_, All), Max),
    (   Max =:= 0
    ->  Turns = []                              % no legal move: pass
    ;   findall(F-Ms, member(Max-(F-Ms), All), Longest),
2       larger_die(Max, Dice, Longest, Legal),
3       sort(1, @<, Legal, Turns)               % one per final position
    ).

2 larger_die(1, [D1, D2], Turns, Big) :-
    D1 =\= D2, B is max(D1, D2),
    include(first_die(B), Turns, Big), Big \== [], !.
larger_die(_, _, Turns, Turns).
first_die(B, _-[m(_, _, B)]).
```

1. Keep only the turns that use the most dice. None at all means pass.
2. If only one die fits and they differ, it must be the larger one.
3. sort/4 keeps one turn per final position: many orders end the same.

EVAL: five features, five weights

```
default_weights(w(1, 0, 4, 3, 2)).

eval(S, _, 1000) :- winner(S, white), !.
eval(S, _, -1000) :- winner(S, black), !.
eval(S, w(WP, WO, WB, WH, WR), V) :-
    features(S, f(Pips, Off, Blots, Home, Bar)),
    V is WP*Pips + WO*Off + WB*Blots + WH*Home + WR*Bar.
```

Pips	Off	Blots	Home	Bar
1	0	4	3	2
difference in pip count: who is ahead in the race	checkers already borne off	lone checkers the rival could hit	points made in your own home board	checkers stuck on the bar

All features are differences, from white's point of view. Without contact between the sides, only the race (pips) counts.

Expectiminimax with a cutoff, in three predicates

```
    % The 21 distinct rolls: doubles 1/36, the others 2/36
1   roll([B, A], P) :-
        between(1, 6, A), between(A, 6, B),
        ( A =:= B -> P is 1/36 ; P is 2/36 ).

    % best_reply(+State, +Player, +Dice, +W, -V):
    %   Player rolls Dice and plays his best move; we stop there and
    %   use EVAL instead of searching deeper (the cutoff).
2   best_reply(S, P, Dice, W, V) :-
        legal_turns(S, P, Dice, Turns),
        (   Turns == []
        ->  eval(S, W, V)
        ;   findall(E, (member(F-_, Turns), eval(F, W, E)), Es),
            ( P == white -> max_list(Es, V) ; min_list(Es, V) )
        ).

    % chance(+State, +Player, +W, -V): the chance node before Player rolls
3   chance(S, P, W, V) :-
        aggregate_all(sum(Pr*Vr), ( roll(D, Pr), best_reply(S, P, D, W, Vr) ), V).
```

1. The chance node's 21 outcomes and their probabilities.
2. The rival plays his best reply, judged by EVAL: here we cut.
3. The chance node is the sum of P(roll) × value.

decide: the move with the best expected value

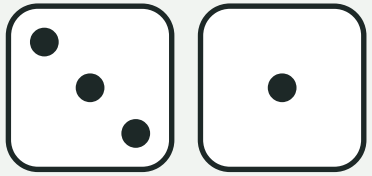
```
1  decide(S, black, Dice, W, Beam, best(V, Ms, F)) :- !,  % decide in the mirror
       mirror(S, M), decide(M, white, Dice, W, Beam, best(V0, Ms0, F0)),
       V is -V0, maplist(mirror_move, Ms0, Ms), mirror(F0, F).
   decide(S, white, Dice, W, Beam, best(V, Ms, F)) :-
       legal_turns(S, white, Dice, Turns), Turns \== [],
2      findall(E-(F0-Ms0), (member(F0-Ms0, Turns), eval(F0, W, E)), Scored),
       keysort(Scored, Up), reverse(Up, Sorted),
       take(Beam, Sorted, Top),
3      findall(V0-(Ms0-F0),
               ( member(_-(F0-Ms0), Top),
                 ( winner(F0, _) -> eval(F0, W, V0) ; chance(F0, black, W, V0) ) ),
               Vs),
4      max_member(V-(Ms-F), Vs).
```

1. Black decides in the mirror, as if it were white.
2. Sort by EVAL and keep the Beam best (12): a second cut.
3. For each, the chance node of black's roll.
4. Take the move with the highest expected value.

Run it

```
$ swipl backgammon.pl
?- initial(S), legal_turns(S, white, [3,1], T), length(T, N).
T = [ … 16 final positions … ],
N = 16.

?- initial(S), decide(S, white, [3,1], best(V, Moves, _)).
V = -0.9166666666666665,
Moves = [m(8, 5, 3), m(6, 5, 1)].
```



8/5 6/5: the agent makes its 5-point, the move backgammon books recommend for an opening 3-1. It took about 0.8 seconds.

Same rules, same EVAL as the web demo

4,000

random positions: Prolog and the page's JavaScript find exactly the same legal moves

2,000

positions: both give exactly the same EVAL value

0

differences, including doubles, the bar, bearing off and turns with no move

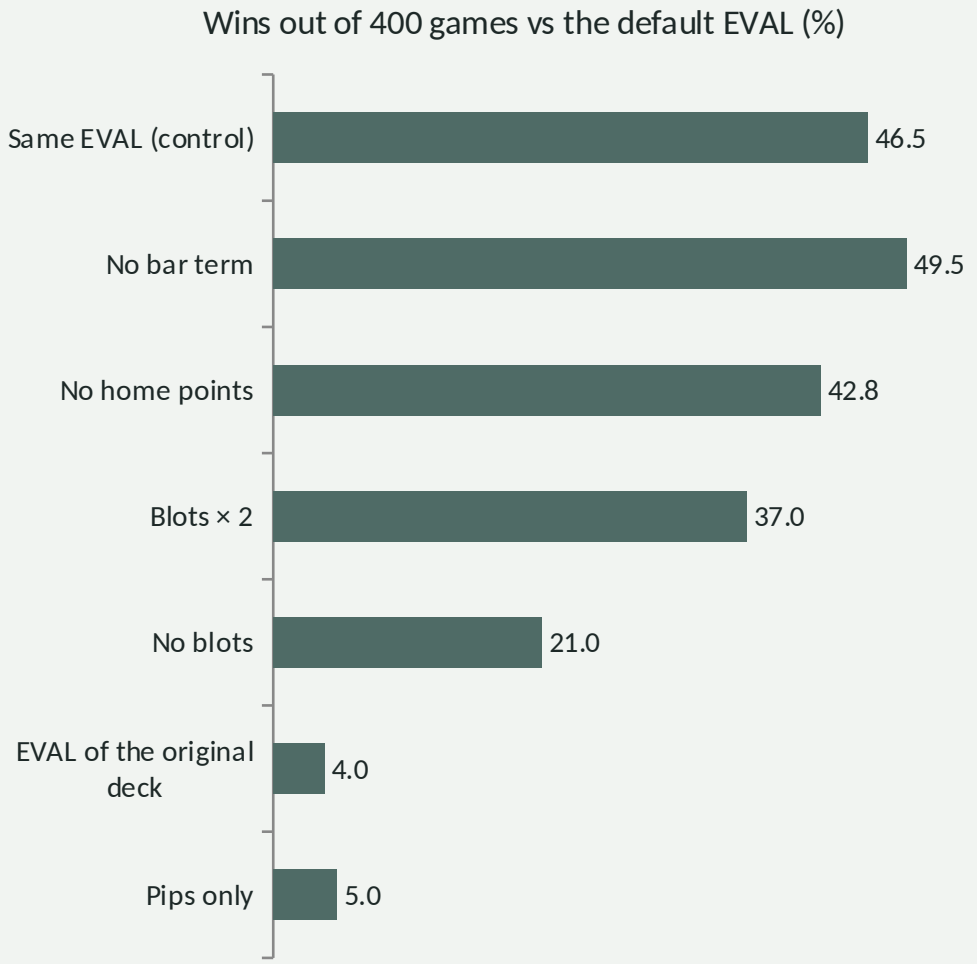
What you play at ferdiex.github.io/backgammon is the same game you can now inspect, query and change in Prolog.

Design your own EVAL

```
?- time(tournament(
       agent(greedy, w(1,0,4,3,2)),
       agent(greedy, w(1,0,0,0,0)),
       100, Wins)).
% 70,378,889 inferences, 7.914 CPU in 8.020 seconds
Wins = 97.
```

A greedy agent plays the move with the best EVAL, without looking ahead: a fast way to compare weights. 100 games take about 8 seconds.

Control ≈ 50%. With 400 games, differences under ±5 points are just luck.



A classic Prolog bug we hit on the way

Our first tournament said white won 200 of 200 games, even against itself.

Before

```
game_loop(S, _, _, _, _, W) :-
    winner(S, W), !.

% called as:
game(AW, AB, First, white)
```

After

```
game_loop(S, _, _, _, _, W) :-
    winner(S, W0), !, W = W0.

% called as:
game(AW, AB, First, Winner),
Winner == white
```

Why: W was already bound to white. When black won, winner(S, white) simply failed, so the loop kept playing until white also bore off its 15 checkers. An output argument bound too early does not raise an error: it changes what the program means.

Predict first, then run

1 Tune one weight

Will doubling the blot weight help? Run a control too.

```
?- tournament(agent(greedy, w(1,0,8,3,2)),
              agent(greedy, w(1,0,4,3,2)), 200, Wins).
```

2 Add a feature

Points made anywhere (2+ checkers). Does it beat the noise?

```
eval(S, w(WP, WO, WB, WH, WR, WM), V) :-
    features(S, f(Pips, Off, Blots, Home, Bar, Made)),
    V is ... + WM*Made.
% in features/2, when there is contact:
    aggregate_all(count, (member(C, Pts), C >= 2), MW),
    aggregate_all(count, (member(C, Pts), C =< -2), MB),
    MadeDiff is MW - MB
```

3 Is looking ahead worth it?

Expectiminimax vs greedy, same EVAL. Slow: try 10 games.

```
?- tournament(agent(expecti(12), w(1,0,4,3,2)),
              agent(greedy, w(1,0,4,3,2)), 10, Wins).
```

Chance changes the max into an average

One chance level plus a good EVAL is enough to play reasonable backgammon. Which EVAL is good is an experimental question.

21 rolls averaged at every chance node

0.45 s per decision in Prolog

4% wins of an EVAL that ignores blots and home points